

Section 5: Challenge EEP1 register multiplication

2026年2月23日 星期一 17:04

5.1 Challenge Overview

The challenge in Lab 2 is to implement additional instruction(s) and/or hardware that speed up EEP1 register multiplication.

5.2 Design Constraints

No more than 64 full adders in total

Credit can be obtained from any of the following:

1. The quality of your solution itself (instruction design, hardware structure, software interface).
2. Speed (in clock cycles) and hardware cost (number of adders) against the "pure software" implementations from Lab 2.
3. Understanding of how your solution can be used in different contexts (for example, implementing signed and unsigned multiplication).

5.3 Design Notes

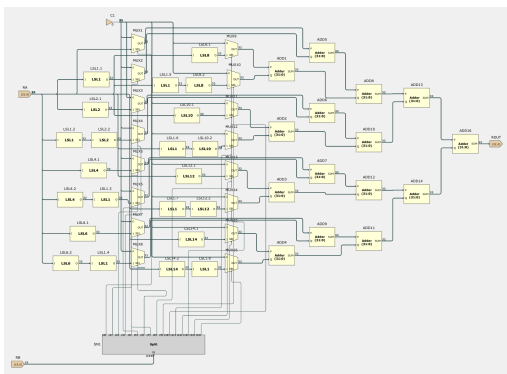
Speeding up EEP1 multiplication generally requires defining one or more additional ALU instructions in the ISA. This is possible because some machine-word encodings are currently unused. Specifically, the standard MOV instruction does not use register c, and the three bits encoding this field are set to zero for a normal MOV.

We choose MOVc1 Ra, Rb as our multiplication assembly language. Once we write this, hardware will run MUL32 in DATAPATH and return the result to Ra, as it is initially a mov instruction, so result can only go to Ra.

Note that for register Ra it is a 16-bit register, but our multiplier can do 32 bits. So even though we obtain a 32-bit result, we can only load high-16 or low-16 into Ra. In my design I choose to load low-16 bits.

5.3.3 Using your new hardware

First we consider a normal multiplier hardware which uses the same logic as the software one we've implemented before.



It satisfies one requirement of the task perfectly, which is the speed. It is very fast and efficient as only needs one cycle to work. It's design is very straightforward:

When corresponding shift bit in RB = 0, we add 0 to result,

When shift bit in RB = 1, we add a shifted RA, and number that shifted depends on which bit it is in RB.

However, it has a big disadvantage. It uses too many adders (much greater than 64) and would cost a lot to build.

As a result, we need to find an alternative way, that sacrifices speed, but uses less adders.

The way I used is called Radix-4 Booth multiplier.

Instead of dealing with RB bit by bit, we deal with it 2 bits at a time, together with one bit from the lower term. As any 16-bit signed binary can be written and arranged as:

For a 16-bit signed decimal B:

$$B = -2^{15} \times B_{15} + 2^{14} \times B_{14} + 2^{13} \times B_{13} + \dots + 2^1 \times B_1 + 2^0 \times B_0$$
$$= 2^{15} \times (-B_{15}) + 2^{14} \times B_{14} + 2^{13} \times B_{13} + \dots + 2^1 \times B_1 + 2^0 \times B_0$$
$$= 2^{15} \times (-2B_{15} + B_{14}) + 2^{14} \times B_{13} + 2^{13} \times B_{12} + \dots + 2^1 \times B_1 + 2^0 \times B_0$$

(We need $2^{15} \times B_{15}$ then)

For a 16-bit signed binary B:

$$B = -2^{15} \times B_{15} + 2^{14} \times B_{14} + 2^{13} \times B_{13} + \dots + 2^1 \times B_1 + 2^0 \times B_0$$
$$= -2^{15} \times B_{15} + 2^{14} \times B_{14} + 2^{13} \times B_{13} + \dots + 2^1 \times B_1 + 2^0 \times B_0$$
$$= -2^{15} \times B_{15} + 2^{14} \times B_{14} + 2^{13} \times B_{13} + \dots + 2^1 \times B_1 + 2^0 \times B_0$$
$$= 2^{15} \times (-2B_{15} + B_{14} + B_{13}) - 2^{14} \times B_{13} + \dots + 2^1 \times B_1 + 2^0 \times B_0$$

Similar to $2^{15} \times (-2B_{15} + B_{14} + B_{13}) - 2^{14} \times B_{13} + \dots + 2^1 \times B_1 + 2^0 \times B_0$

$$2^{15} \times (-2B_{15} + B_{14} + B_{13}) - 2^{14} \times B_{13} + \dots + 2^1 \times B_1 + 2^0 \times B_0$$

So $B = 2^{15} \times (-2B_{15} + B_{14} + B_{13}) + 2^{14} \times (-2B_{14} + B_{13} + B_{12}) + 2^{13} \times (-2B_{13} + B_{12} + B_{11}) + \dots + 2^1 \times (-2B_1 + B_0 + B_{-1}) + 2^0 \times (-2B_0 + B_{-1} + B_{-2})$

Note we have an addition term B_{-1} as long as we set it as 0. $B_{-1} + B_{-1} = B + 0 = B$

This won't be a problem

We call a $-2b_{n-1} + b_{n-2} + b_{n-3}$ Radix-4 index.
 For an index: min = $-2+0+0 = -2$;
 Max = $0+1+1 = 2$.
 So we have $-2; -1; 0; 1; 2$.

$$RB = (-2b_{n-1} + b_{n-2} + b_{n-3})2^{n-2} + (-2b_{n-2} + b_{n-3} + b_{n-4})2^{n-3} + \dots + (-2b_1 + b_0 + b_{-1})2^0$$

$$\therefore RA \cdot RB = 2^{n-2} \cdot RA \cdot (-2b_{n-1} + b_{n-2} + b_{n-3}) + 2^{n-3} \cdot RA \cdot (-2b_{n-2} + b_{n-3} + b_{n-4}) + \dots + 2^0 \cdot RA \cdot (-2b_1 + b_0 + b_{-1})$$

It is an addition of 8 terms, so we can give MOVCI with about 8 cycles, to accumulate to final result, so that we only need one 32-bit adder.

And the path is like this:

1. For RB, every 3 bits (overlap between adjacent ones) we calculate a Radix-4 index.
2. For RA, LSL by $2(N-1)$ at Nth cycle of MOVCI, so that after 8 cycles, it goes from LSL0 to LSL14.
3. Multiply shifted RA by index, it is among $-2A, -A, 0, A, 2A$.
4. Clear all registers and load and disable all counters.

We don't do Radix-8 because Radix-8 involves $\pm 3RA$, which will need additional 32-bit adder. That beyond limit but could only save 4 more cycles, so it's not a good idea.

My total hardware is like this:

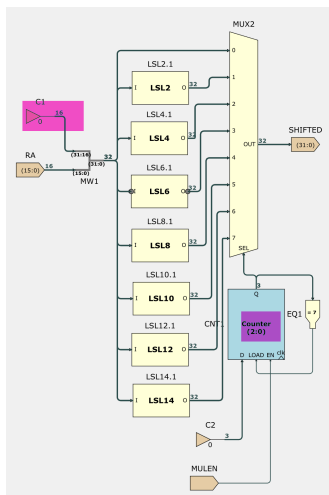


Fig1. LSL sel for RA

First EXT RA to 32 bits make it be able to shift up to 14 bits.
 We see that a counter controls MUX to select LSL2(N-1) of RA in Nth cycle.

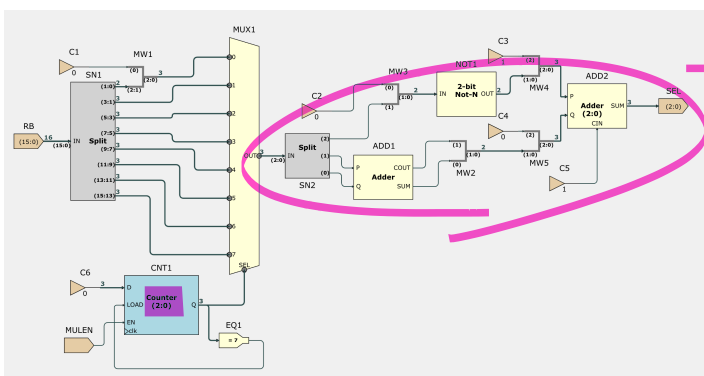
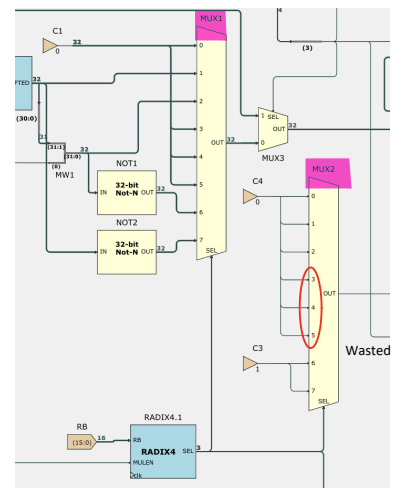


Fig2. Radix-4 index

Index calculation



Also a counter control MUX to select corresponding bits of RB, and they do the index calculation.

Because $-2; -1; 0; 1; 2$ would need a 8-input MUX to choose and produce 3 wasted import, and our b_{n-1} ; b_{n-2} ; b_{n-3} would give 8 combinations. So we can just list all possibilities and make full use of MUX, that would save adders. Like following:

	A	B	C	D	E
1	b_{n-1}	b_{n-2}	b_{n-3}	Port	Index
2	0	0	0	0	0
3	0	0	1	1	1
4	0	1	0	2	1
5	0	1	1	3	2
6	1	0	0	4	-2
7	1	0	1	5	-1

Now the design is clear.

One thing needs to note is that I set 9 cycles to finish multiplication because we need an additional cycle to reset all counter to 0, at the same time clear values in register so that it can be reused afterwards. But we have to write 0 into adder in that cycle so that our result doesn't change any more, and that's what MUX7 does, it has to be consistent with CIN in MUX2

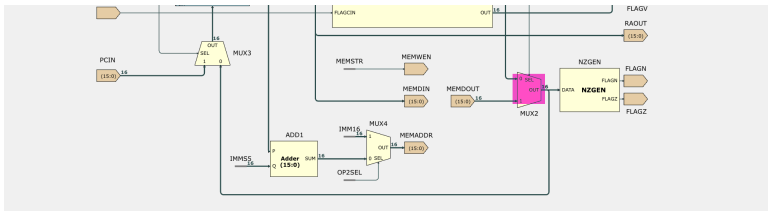


Fig5. DATAPATH implementation

Design in DATAPATH is straightforward, we just add a MUX. When our MULEN is on, we choose to write result from MUL32 to register instead of ALU.

I delayed MULEN for one cycle because when PC runs again, MOV1 still needs a cycle to move, so we still have to select result from MUL32, otherwise the result will be lost and we will just do a normal MOV instruction.

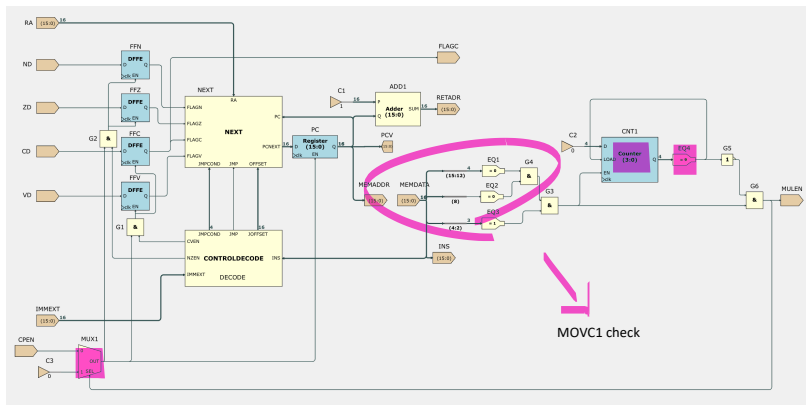


Fig6. CONTROLPATH implementation

In CONTROLPATH we do a check for MOVC1, which has assembly language:

How to read EEP1 Assembly Language

[illegible]

Once true, we create a MULEN which will be high for 9 cycles, and also freeze PC for 9 cycles to give MUL32 enough time to implement multiplication.

That's all for MUL32 and now we run a test:

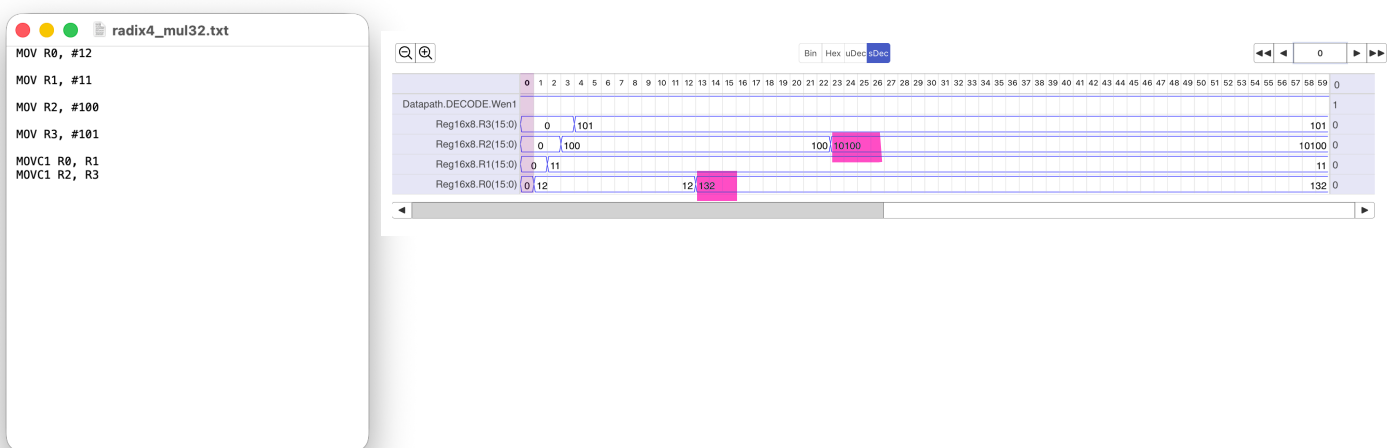


Fig7. Simple test

Both cases correct, and we also proved that this multiplier can be reused.